

EX NO: 9

CREATE DOCUMENT, COLUMN AND GRAPH BASED DATA USING NOSQL DATABASE TOOLS

DATE:

AIM:

To Create document, column and graph based data using NOSQL database tools.

Key Value Pair Based

- Data is stored in key/value pairs. It is designed in such a way to handle lots of data and heavy load.
- Key-value pair storage databases store data as a hash table where each key is unique, and the value can be a JSON, BLOB(Binary Large Objects), string, etc.
- For example, a key-value pair may contain a key like "Website" associated with a value like "Guru99".

Key	Value
Name	Joe Bloggs
Age	42
Occupation	Stunt Double
Height	175cm
Weight	77kg

- It is one of the most basic NoSQL database example. This kind of NoSQL database is used as a collection, dictionaries, associative arrays, etc. Key value stores help the developer to store schema-less data. They work best for shopping cart contents.
- Redis, Dynamo, Riak are some NoSQL examples of key-value store DataBases. They are all based on Amazon's Dynamo paper.
-

Column-based

- Column-oriented databases work on columns and are based on BigTable paper by Google. Every column is treated separately. Values of single column databases are stored contiguously

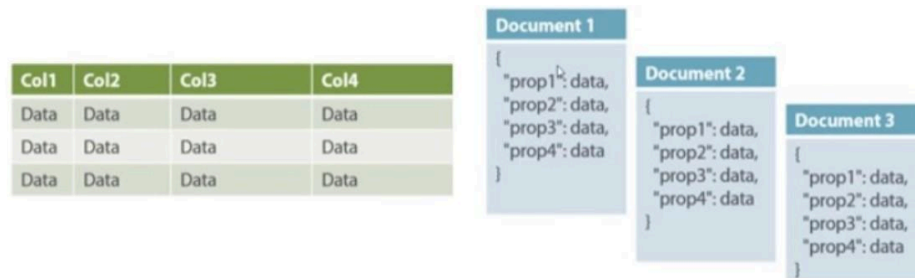
ColumnFamily			
Row Key	Column Name		
	Key	Key	Key
	Value	Value	Value
	Column Name		
	Key	Key	Key
	Value	Value	Value

- Column based NoSQL database.

- They deliver high performance on aggregation queries like SUM, COUNT, AVG, MIN etc. as the data is readily available in a column.
- Column-based NoSQL databases are widely used to manage data warehouses, business intelligence, CRM, Library card catalogs,
- HBase, Cassandra, HBase, Hypertable are NoSQL query examples of column based database.

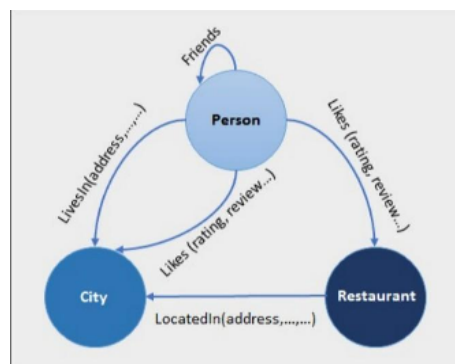
Document-Oriented

- Document-Oriented NoSQL DB stores and retrieves data as a key value pair but the value part is stored as a document. The document is stored in JSON or XML formats. The value is understood by the DB and can be queried.



Graph-Based

- A graph type database stores entities as well the relations amongst those entities. The entity is stored as a node with the relationship as edges. An edge gives a relationship between nodes. Every node and edge has a unique identifier.



PROGRAM

Select database and Insert one document

use college

OUTPUT:

switched to db library

```
db.students.insertOne({  
  student_id: 101,  
  name: "Alice",  
  age: 20,  
  course: "Computer Science",  
  city: "Chennai"  
})
```

OUTPUT:

```
{ acknowledged: true,insertedId: ObjectId('...') }
```

Insert many document

```
db.students.insertMany([ {  
  student_id: 102,  
  name: "Bob",  
  age: 21,  
  course: "Information Technology",  
  city: "Madurai"  
},  
{  
  student_id: 103,  
  name: "Charlie",  
  age: 20,  
  course: "Electronics",  
  city: "Coimbatore"
```

```
}})
```

Display all documents

```
db.students.find()
```

OUTPUT:

```
[{
  _id: ObjectId('...'),
  student_id: 101,
  name: "Alice",
  age: 20,
  course: "Computer Science",
  city: "Chennai"
},
{
  _id: ObjectId('...'),
  student_id: 102,
  name: "Bob",
  age: 21,
  course: "Information Technology",
  city: "Madurai"
},
{
  _id: ObjectId('...'),
  student_id: 103,
  name: "Charlie",
  age: 20,
  course: "Electronics",
  city: "Coimbatore"
}]
```

RESULT

Thus the create document, column and graph based data using nosql database tools was executed.

PROGRAM

LIBRARY MANAGEMENT SYSTEM

Select or Create Database

use library

OUTPUT:

switched to db library

Insert Authors

```
db.authors.insertMany([
  {
    AuthorID: 1,
    FirstName: "George",
    LastName: "Orwell"
  },
  {
    AuthorID: 2,
    FirstName: "Aldous",
    LastName: "Huxley"
  },
  {
    AuthorID: 3,
    FirstName: "J.K.",
    LastName: "Rowling"
  }
])
```

OUTPUT:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('...'),
    '1': ObjectId('...'),
    '2': ObjectId('...')
  }
}
```

Insert Books

```
db.books.insertMany([
  {
    BookID: 1,
    Title: "1984",
    Genre: "Dystopian",
```

```

PublicationYear: 1949,
Authors: [1]
},
{
BookID: 2,
Title: "Brave New World",
Genre: "Dystopian",
PublicationYear: 1932,
Authors: [2]
},
{
BookID: 3,
Title: "Harry Potter and the Sorcerer's Stone",
Genre: "Fantasy",
PublicationYear: 1997,
Authors: [3]
}
])

```

OUTPUT:

```

{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('...'),
    '1': ObjectId('...'),
    '2': ObjectId('...')
  }
}

```

Insert Borrowers

```

db.borrowers.insertMany([
  {
    BorrowerID: 1,
    FirstName: "John",
    LastName: "Doe",
    MembershipDate: new Date("2023-01-01")
  },
  {
    BorrowerID: 2,
    FirstName: "Jane",
    LastName: "Smith",
    MembershipDate: new Date("2023-02-15")
  }
])

```

OUTPUT:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('...'),
    '1': ObjectId('...')
  }
}
```

Step 5: Insert Borrowed Books

```
db.borrowedBooks.insertMany([
  {
    BorrowerID: 1,
    BookID: 1,
    BorrowedDate: new Date("2023-03-01"),
    ReturnDate: new Date("2023-03-15")
  },
  {
    BorrowerID: 2,
    BookID: 3,
    BorrowedDate: new Date("2023-03-05"),
    ReturnDate: new Date("2023-03-20")
  }
])
```

OUTPUT:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('...'),
    '1': ObjectId('...')
  }
}
```

Update Data

Change George Orwell's last name to Smith.

```
db.authors.updateOne(
  { AuthorID: 1 },
  { $set: { LastName: "Smith" } }
)
```

OUTPUT:

```
{
  acknowledged: true,
  matchedCount: 1,
```

```
modifiedCount: 1
}
```

Step 7: Query / Read Data

Display all authors:

```
db.authors.find().pretty()
```

OUTPUT:

```
[
  {
    _id: ObjectId('...'),
    AuthorID: 1,
    FirstName: "George",
    LastName: "Smith"
  },
  {
    _id: ObjectId('...'),
    AuthorID: 2,
    FirstName: "Aldous",
    LastName: "Huxley"
  },
  {
    _id: ObjectId('...'),
    AuthorID: 3,
    FirstName: "J.K.",
    LastName: "Rowling"
  }
]
```

Display all books:

```
db.books.find().pretty()
```

OUTPUT:

```
[
  {
    _id: ObjectId('...'),
    BookID: 1,
    Title: "1984",
    Genre: "Dystopian",
    PublicationYear: 1949,
    Authors: [1]
  },
  {
    _id: ObjectId('...'),
```

```
BookID: 2,  
Title: "Brave New World",  
Genre: "Dystopian",  
PublicationYear: 1932,  
Authors: [2]  
},  
{  
  _id: ObjectId('...'),  
  BookID: 3,  
  Title: "Harry Potter and the Sorcerer's Stone",  
  Genre: "Fantasy",  
  PublicationYear: 1997,  
  Authors: [3]  
}  
]
```

Display all borrowers:

```
db.borrowers.find().pretty()
```

OUTPUT:

```
[  
  {  
    _id: ObjectId('...'),  
    BorrowerID: 1,  
    FirstName: "John",  
    LastName: "Doe",  
    MembershipDate: ISODate("2023-01-01T00:00:00.000Z")  
  },  
  {  
    _id: ObjectId('...'),  
    BorrowerID: 2,  
    FirstName: "Jane",  
    LastName: "Smith",  
    MembershipDate: ISODate("2023-02-15T00:00:00.000Z")  
  }  
]
```

Display all borrowed books:

```
db.borrowedBooks.find().pretty()
```

OUTPUT:

```
[  
  {  
    _id: ObjectId('...'),  
    BorrowerID: 1,
```

```
BookID: 1,  
BorrowedDate: ISODate("2023-03-01T00:00:00.000Z"),  
ReturnDate: ISODate("2023-03-15T00:00:00.000Z")  
},  
{  
  _id: ObjectId('...'),  
  BorrowerID: 2,  
  BookID: 3,  
  BorrowedDate: ISODate("2023-03-05T00:00:00.000Z"),  
  ReturnDate: ISODate("2023-03-20T00:00:00.000Z")  
}  
]
```

Delete Data

Delete author whose AuthorID is 1.

```
db.authors.deleteOne(  
  { AuthorID: 1 }  
)
```

OUTPUT:

```
{  
  acknowledged: true,  
  deletedCount: 1  
}
```

Verify Deletion

```
db.authors.find().pretty()
```

OUTPUT:

```
[  
  {  
    _id: ObjectId('...'),  
    AuthorID: 2,  
    FirstName: "Aldous",  
    LastName: "Huxley"  
  },  
  {  
    _id: ObjectId('...'),  
    AuthorID: 3,  
    FirstName: "J.K.",  
    LastName: "Rowling"  
  }  
]
```